

An improved list decoding algorithm for the second order Reed–Muller codes and its applications

Rafaël Fourquet · Cédric Tavernier

Received: 31 May 2007 / Revised: 16 January 2008 / Accepted: 25 January 2008 /
Published online: 19 March 2008
© Springer Science+Business Media, LLC 2008

Abstract We propose an algorithm which is an improved version of the Kabatiansky–Tavernier list decoding algorithm for the second order Reed–Muller code $RM(2, m)$, of length $n = 2^m$, and we analyse its theoretical and practical complexity. This improvement allows a better theoretical complexity. Moreover, we conjecture another complexity which corresponds to the results of our simulations. This algorithm has the strong property of being deterministic and this fact drives us to consider some applications, like determining some lower bounds concerning the covering radius of the $RM(2, m)$ code.

Keywords List decoding · Reed–Muller codes · Covering radius · Boolean functions · Cryptography

AMS Classifications 94B05 · 94B35 · 94B65 · 94A60

1 Introduction

Introduced by Elias [1] 50 years ago, the concept of list decoding has been recently revived thanks to Sudan's discovery [2] of efficient list decoding algorithms for Reed–Solomon (RS) codes. Despite that there are similarities between Reed–Solomon and Reed–Muller (RM) codes, no efficient list decoding algorithm for RM codes was known until very recently.

This article is a rewritten and completed version of Fourquet R., Tavernier C.: List decoding of second order Reed–Muller codes and its covering radius implications, Workshop on Coding and Cryptography 2007, pp. 147–156.

R. Fourquet (✉)
Département de Mathématiques, Université Paris 8 Saint-Denis (MAATICA), 2, rue de la Liberté,
Saint-Denis cedex 93526, France
e-mail: raphael.fourquet@gmail.com

C. Tavernier
THALES Communications, Colombes, France
e-mail: tavernier.cedric@gmail.com

Goldreich and Levin [3] have suggested a probabilistic algorithm which works in the worst case for the first order binary Reed–Muller code $\text{RM}(1, m)$. Subsequently, Goldreich et al. [4] have generalized this algorithm over a finite field and suggested an extension for any order over a large finite field. Then, deterministic algorithms which work for any order have been suggested by Pellikaan and Wu in [5], and subsequently by Kabatiansky and Tavernier in [6] for the second order and by Dumer et al. [7] for any order. This last algorithm considerably improves the algorithm of [5] by correcting up to the Johnson bound with an almost linear complexity. Here, we are interested in decoding the $\text{RM}(2, m)$ code beyond minimal distance, and we propose an improvement of the algorithm of [6]. This algorithm is correlated with the techniques used in [3] and [4]. Nevertheless, these techniques cannot be derived in a simple way. The algorithm of [3] is probabilistic and works for the first order Reed–Muller code, whereas we propose a deterministic algorithm for the second order Reed–Muller code. Many difficult problems appear as soon as we want to decode beyond the Johnson bound.

There is a natural one to one correspondence between the second order Reed–Muller code $\text{RM}(2, m)$ and the set of m -variable quadratic Boolean functions. A codeword of $\text{RM}(2, m)$ can be represented by the evaluation of a quadratic Boolean function over its definition set. Then we can define the Hamming distance d between a Boolean function, which corresponds to the received vector, and the set of quadratic functions $\text{RM}(2, m)$.

The covering radius $\rho(2, m)$ of the second order Reed–Muller code $\text{RM}(2, m)$ is defined as $\rho(2, m) = \max_{y \in \mathcal{B}_m} d(y, \text{RM}(2, m))$, where $\mathcal{B}_m$ is the set of m -variable Boolean functions. This quantity is still unknown for $m \geq 7$. However, it has been recently upper bounded in [8]. Some lower bounds are also known (see [9, 10]). In this article, we are going to test some cryptographic Boolean functions in order to improve the lower bound of the covering radius up to $m = 12$.

We also propose some simulations in the binary symmetric channel. It appears, according to our simulation results, that the performances of this algorithm are comparable to the performances of the Ilya Dumer algorithm (see [11]) for a “reasonable” quantity of errors. We will not propose a proof, nevertheless it could be a starting point for a next work.

2 The second order Reed–Muller codes

2.1 Basic definitions and notations

We denote by $\mathbb{F}_2$ the field with two elements, and by $\mathcal{B}_m$, for some positive integer m , the set of all Boolean functions $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$. Such a Boolean function f can be uniquely represented as an m -variable polynomial over $\mathbb{F}_2$ (the *Algebraic Normal Form*). The *algebraic degree* of f is the degree of this polynomial. An *affine* (resp. *quadratic*) Boolean function is of algebraic degree at most 1 (resp. 2). A Boolean function f can also be identified with the binary vector, of length 2^m consisting of all values $f(x)$, $x \in \mathbb{F}_2^m$:

$$f = (f(0, 0, \dots, 0), f(1, 0, \dots, 0), f(0, 1, \dots, 0), f(1, 1, \dots, 0), \dots, f(1, 1, \dots, 1)).$$

Conversely, any binary vector of length 2^m will also be considered as a Boolean function in $\mathcal{B}_m$. From now, m will denote a fixed positive integer, and we set $n = 2^m$. Let $f = (f_1, \dots, f_n)$ and $g = (g_1, \dots, g_n)$ be two binary vectors of length n , corresponding to m -variable Boolean functions. The *Hamming distance* $d(f, g)$ between these vectors is the number of coordinates where f and g differ:

$$d(f, g) = |\{i \in [1, \dots, n], f_i \neq g_i\}|.$$

The *Hamming weight* $\text{wt}(f)$ of f is the distance between f and the null vector. We will also denote by f_χ (the font is changed) the vector of $\{-1, 1\}^n$ corresponding to the function $(-1)^f$ (the sign function of f), that is, $f_\chi = ((-1)^{f_1}, \dots, (-1)^{f_n})$.

For any Boolean function $f \in \mathcal{B}_m$, we denote by $\mathcal{F}(f)$ the following character sum, related to the Walsh transform of f :

$$\mathcal{F}(f) = \sum_{x \in \mathbb{F}_2^m} (-1)^{f(x)} = n - 2\text{wt}(f). \quad (1)$$

Note that $\mathcal{F}(f + 1) = -\mathcal{F}(f)$. We will also use in this article the usual scalar product, which is very adapted to our problems. Let $x = (x_1, \dots, x_n)$ and $y = (y_1, \dots, y_n)$ be two vectors of $\mathbb{Z}^n$. We denote by $\langle x, y \rangle$ the scalar product of x and y , defined by

$$\langle x, y \rangle = \sum_{i=1}^n x_i y_i.$$

The usual Euclidean norm satisfies the relation $\|x\|^2 = \langle x, x \rangle$, and the useful Cauchy–Schwartz inequality gives $\langle x, y \rangle^2 \leq \|x\|^2 \cdot \|y\|^2$.

We also have, for $f, g \in \mathcal{B}_m$:

$$\begin{aligned} \langle f_\chi, g_\chi \rangle &= \mathcal{F}(f + g) = n - 2d(f, g), \\ \|f_\chi\|^2 &= n. \end{aligned} \quad (2)$$

Let us now recall the definition of the Reed–Muller codes of orders 1 and 2. The first-order binary Reed–Muller code $\text{RM}(1, m)$, of length n , consists of vectors $a = (\dots, a(x_1, \dots, x_m), \dots)$, where

$$a(x) = a(x_1, \dots, x_m) = a_0 + a_1 x_1 + \dots + a_i x_i + \dots + a_m x_m, \quad a_i \in \mathbb{F}_2, \quad 0 \leq i \leq m,$$

is any affine Boolean function, and where $(x_1, \dots, x_m)$ runs over all 2^m points of the Boolean cube of dimension m . This code has dimension $m + 1$ and minimal distance $n/2$. We will also denote by $\text{RM}(1, m)^\#$ the subset of $\text{RM}(1, m)$ consisting of all linear functions (with a null constant term a_0).

Now, the second-order binary Reed–Muller code $\text{RM}(2, m)$, of length n , consists of vectors $q = (\dots, q(x_1, \dots, x_m), \dots)$, where

$$q(x) = q(x_1, \dots, x_m) = a_0 + \sum_{i=1}^m a_i x_i + \sum_{1 \leq i < j \leq m} q_{i,j} x_i x_j \quad (3)$$

is a quadratic Boolean function, with $a_i, q_{i,j} \in \mathbb{F}_2$. This code has dimension $1 + \binom{m}{1} + \binom{m}{2}$ and minimal distance $n/4$. We will also denote by $\text{RM}(2, m)^\#$ the subset of $\text{RM}(2, m)$ consisting of all quadratic functions with a null affine component ($a_0 = a_1 = \dots = a_m = 0$).

We will use in this article a representation for the elements of the $\text{RM}(2, m)$ code that corresponds to our algorithm. A quadratic Boolean function $q \in \text{RM}(2, m)$ can be written in the following way:

$$q(x_1, \dots, x_m) = Q_1(x_1, \dots, x_m) + \sum_{i=2}^m x_i Q_i(x_1, \dots, x_{i-1}), \quad (4)$$

where $Q_1 \in \text{RM}(1, m)$ is an affine Boolean function and $Q_i \in \text{RM}(1, i - 1)^\#$ is a linear Boolean function for $2 \leq i \leq m$.

2.2 Deterministic list decoding

The notion of list decoding was introduced in 1957 by Elias in [1]. By definition, a list decoding algorithm of decoding radius T should produce, for any received vector y , the list $L_T(y) = \{c \in C : d(y, c) \leq T\}$ of all vectors c belonging to a code C which are at distance at most T apart from y . In our case, we consider a decoding radius of the form $T = n(1/2 - \varepsilon)$, where ε is a real number such that $0 < \varepsilon \leq 1/2$, and we denote by

$$\mathcal{L}_{y,\varepsilon} = \left\{ q \in \text{RM}(2, m) : d(y, q) \leq n \left(\frac{1}{2} - \varepsilon \right) \right\}$$

the desired list, for a received vector y . In terms of character sum, according to relation (2), the list is also defined by

$$\mathcal{L}_{y,\varepsilon} = \{q \in \text{RM}(2, m) : \mathcal{F}(y + q) \geq 2n\varepsilon\}.$$

By deterministic list decoding, we mean that the algorithm must always return the complete list $\mathcal{L}_{y,\varepsilon}$, with a null probability of failure. Our method consists in reconstructing recursively the affine (or linear) coefficients Q_i , $1 \leq i \leq m$, according to representation (4), of the solutions $q \in \mathcal{L}_{y,\varepsilon}$.

2.3 Weight enumerator and bound on the size of the list.

The problem that we want to solve is more pertinent when the size of the list $\mathcal{L}_{y,\varepsilon}$ is still polynomial in the length $n = 2^m$ of the code. Hence, we are going to give some results in order to bound this size, but let us first recall the following result concerning the weight enumerator of the second order Reed–Muller code.

Theorem 1 ([12]) *Let A_i be the number of codewords of weight i in the $\text{RM}(2, m)$ code. Then A_i is given by $A_i = 0$ unless $i = 2^{m-1}$ or $2^{m-1} \pm 2^{m-1-j}$, for some j , $0 \leq j \leq \lfloor m/2 \rfloor$, and*

$$\begin{aligned} A_0 &= A_{2^m} = 1, \\ A_{2^{m-1} \pm 2^{m-1-j}} &= 2^{j(j+1)} \prod_{i=1}^j \frac{(2^{m-2i+2} - 1)(2^{m-2i+1} - 1)}{4^i - 1}, \quad 1 \leq j \leq \lfloor m/2 \rfloor, \\ A_{2^{m-1}} &= 2^{1+m(m+1)/2} - 2 \sum_{j=0}^{\lfloor m/2 \rfloor} A_{2^{m-1} - 2^{m-1-j}}. \end{aligned}$$

Corollary 1 *The quantity $A_{2^{m-1} \pm 2^{m-1-j}}$, for $1 \leq j \leq \lfloor m/2 \rfloor$, is upper bounded by $2^{-2j^2+2j} n^{2j}$.*

Proof $\prod_{i=1}^j \frac{(2^{m-2i+2}-1)(2^{m-2i+1}-1)}{4^i-1} \leq \frac{2^{2jm} \prod_{i=1}^j 2^{-4i+3}}{\prod_{i=1}^j (4^i-1)} \leq 2^{-j(j+1)-2j^2+2j} n^{2j}$. The second inequality comes from that $2^{2jm} \prod_{i=1}^j 2^{-4i+3} = n^{2j} 2^{j-2j^2}$ on the one hand, and that $\prod_{i=1}^j (4^i - 1) > \prod_{i=1}^j 2^{2i-1} = 2^{j^2}$ on the other hand. $\square$

The well known Johnson bound (see [13]) gives us a polynomial upper bound on the size of $\mathcal{L}_{y,\varepsilon}$ when $\varepsilon > \frac{1}{2\sqrt{2}}$. In fact, it is possible to obtain some polynomial bounds for smaller values of ε .

Theorem 2 ([6]) *Let j be an integer such that $1 \leq j \leq \lfloor m/2 \rfloor$. Then, for any received vector $y \in \mathbb{F}_2^m$ and for any $\varepsilon > 2^{-1-j/2}$, we have*

$$|\mathcal{L}_{y,\varepsilon}| \leq \frac{\sum_{i=0}^{j-1} A_{2^{m-1}-2^{m-1-i}} (2^{-i} - 2^{-j})}{4\varepsilon^2 - 2^{-j}}.$$

In particular, the case $j = 1$ corresponds to the Johnson bound.

Proof Let $L = |\mathcal{L}_{y,\varepsilon}|$. By definition of $\mathcal{L}_{y,\varepsilon}$, we have $2n\varepsilon L \leq \sum_{q \in \mathcal{L}_{y,\varepsilon}} \mathcal{F}(y + q) = \left\langle y_\chi, \sum_{q \in \mathcal{L}_{y,\varepsilon}} q_\chi \right\rangle$. Hence, using the Cauchy–Schwartz inequality, we get

$$4n^2\varepsilon^2 L^2 \leq \left\langle y_\chi, \sum_{q \in \mathcal{L}_{y,\varepsilon}} q_\chi \right\rangle^2 \leq \underbrace{\|y_\chi\|^2}_n \times \left\| \sum_{q \in \mathcal{L}_{y,\varepsilon}} q_\chi \right\|^2,$$

which implies that

$$4n\varepsilon^2 L^2 \leq \left\langle \sum_{q \in \mathcal{L}_{y,\varepsilon}} q_\chi, \sum_{q \in \mathcal{L}_{y,\varepsilon}} q_\chi \right\rangle. \quad (5)$$

Denoting, for every $q \in \mathcal{L}_{y,\varepsilon}$, by $E(q)$ the set $\{p + q \in \text{RM}(2, m) : p \in \mathcal{L}_{y,\varepsilon}\}$ of size L , we get

$$\mathcal{S}(y) := \left\langle \sum_{q \in \mathcal{L}_{y,\varepsilon}} q_\chi, \sum_{q \in \mathcal{L}_{y,\varepsilon}} q_\chi \right\rangle = \sum_{q \in \mathcal{L}_{y,\varepsilon}} \sum_{f \in E(q)} \sum_{x \in \mathbb{F}_2^m} (-1)^{f(x)},$$

with $\sum_{x \in \mathbb{F}_2^m} (-1)^{f(x)} = \mathcal{F}(f) = n - 2\text{wt}(f)$. In order to upper bound $\mathcal{S}(y)$, we will assume that for any $q \in \mathcal{L}_{y,\varepsilon}$, the set $E(q)$ contains all codewords of weight less than $2^{m-1} - 2^{m-j}$ (the worst case): the number of such codewords is $\sum_{i=0}^{j-1} A_{2^{m-1}-2^{m-1-i}}$, and for $0 \leq i < j$ and $f \in E(q)$ of weight $2^{m-1} - 2^{m-1-i}$, we have $\mathcal{F}(f) = n2^{-i}$. Then, the $L - \sum_{i=0}^{j-1} A_{2^{m-1}-2^{m-1-i}}$ remaining codewords in $E(q)$ have a weight strictly greater than $2^{m-1} - 2^{m-j}$, that is, at least $2^{m-1} - 2^{m-j-1}$: for such a codeword f , we have $\mathcal{F}(f) \leq n2^{-j}$. Then, for any j such that $1 \leq j \leq \lfloor m/2 \rfloor$, we have

$$\mathcal{S}(y) \leq nL \sum_{i=0}^{j-1} 2^{-i} A_{2^{m-1}-2^{m-1-i}} + nL2^{-j} \left(L - \sum_{i=0}^{j-1} A_{2^{m-1}-2^{m-1-i}} \right),$$

and finally, with (5), we get that $L \leq \frac{\sum_{i=0}^{j-1} A_{2^{m-1}-2^{m-1-i}} (2^{-i} - 2^{-j})}{4\varepsilon^2 - 2^{-j}}$. $\square$

3 Deterministic list decoding algorithm for the second order Reed–Muller codes

We shall describe in this section an algorithm (called “*sums*-algorithm”) whose idea is to construct recursively the quadratic functions belonging to the list $\mathcal{L}_{y,\varepsilon} = \{q \in \text{RM}(2, m) : \mathcal{F}(y + q) \geq 2n\varepsilon\}$. This algorithm uses the classical “divide and conquer” method.

Let us first introduce a useful notation: for $f \in \mathcal{B}_m$, $v \in [1, \dots, m]$ and $s \in \mathbb{F}_2^{m-v}$, let $f_s \in \mathcal{B}_v$ be the restriction of f to the “facet”

$$S_s := \{(x, s) \in \mathbb{F}_2^m \mid x \in \mathbb{F}_2^v\},$$

that is, $f_s(x) = f(x, s)$. For the sake of compactness, if $s = (s_1, s_2, \dots, s_{m-\nu}) \in \mathbb{F}_2^{m-\nu}$ and $\delta \in \mathbb{F}_2$, we will denote by “ δs ”, that is “ $0s$ ” or “ $1s$ ”, the vector $(\delta, s_1, s_2, \dots, s_{m-\nu})$ in $\mathbb{F}_2^{m-\nu+1}$. Let us note that, for $s \in \mathbb{F}_2^{m-\nu}$, we have

$$\mathcal{F}(f) = \sum_{s \in \mathbb{F}_2^{m-\nu}} \sum_{x \in \mathbb{F}_2^\nu} (-1)^{f_s(x)} = \sum_{s \in \mathbb{F}_2^{m-\nu}} \mathcal{F}(f_s), \quad (6)$$

with

$$\mathcal{F}(f_s) = \sum_{x \in \mathbb{F}_2^{\nu-1}} (-1)^{f_s(x,0)} + (-1)^{f_s(x,1)} = \mathcal{F}(f_{0s}) + \mathcal{F}(f_{1s}). \quad (7)$$

The proposed *sums*-algorithm will exploit the relation (6), with $f = y + q$, by computing an upper bound of the quantity $\mathcal{F}(q_s + y_s)$. The following definition allows to give a nice description of the restriction q_s of q to a given facet S_s .

Definition 1 Let $q(x_1, \dots, x_m) = Q_1(x_1, \dots, x_m) + \sum_{i=2}^m x_i Q_i(x_1, \dots, x_{i-1}) \in \text{RM}(2, m)$ be a quadratic Boolean function. For $2 \leq \nu \leq m$, we define the ν -th prefix $q^\nu \in \text{RM}(2, \nu)^\#$ of q as the quadratic part of q depending only on the first ν variables:

$$q^\nu(x_1, \dots, x_\nu) = x_2 Q_2(x_1) + \dots + x_\nu Q_\nu(x_1, \dots, x_{\nu-1}).$$

At the ν -th step of the algorithm, we will determine a list $\mathcal{L}_{y,\varepsilon}^\nu$ of candidates which “could”, but may not, coincide with the ν -th prefix of an element $q \in \mathcal{L}_{y,\varepsilon}$.

By definition, for $\nu \leq m$ and for $s \in \mathbb{F}_2^{m-\nu}$, there exists an affine function $l_{q,s} \in \text{RM}(1, \nu)$ such that $q_s = q^\nu + l_{q,s}$, where $l_{q,s}$ is the restriction to S_s of the function $Q_1 + \sum_{i|s_i=1} Q_{\nu+i}$. As a consequence we can rewrite (6):

$$\mathcal{F}(y + q) = \sum_{s \in \mathbb{F}_2^{m-\nu}} \mathcal{F}(y_s + q^\nu + l_{q,s}). \quad (8)$$

Now, for each $s \in \mathbb{F}_2^{m-\nu}$, we have:

$$\mathcal{F}(y_s + q^\nu + l_{q,s}) \leq \max_{l \in \text{RM}(1, \nu)} \mathcal{F}(y_s + q^\nu + l).$$

Finally, we deduce that if $q \in \mathcal{L}_{y,\varepsilon}$, then:

$$\Gamma_y^\nu(q) := \sum_{s \in \mathbb{F}_2^{m-\nu}} \max_{l \in \text{RM}(1, \nu)} \mathcal{F}(y_s + q^\nu + l) \geq \mathcal{F}(y + q) \geq 2n\varepsilon. \quad (9)$$

The key point is that $\Gamma_y^\nu(q)$ depends only on the prefix q^ν of q . This means that a function $q^\nu \in \text{RM}(2, \nu)^\#$ could be the ν -th prefix of a solution $q \in \mathcal{L}_{y,\varepsilon}$ only if it satisfies the “ Γ -criterion” implied by (9), namely only if $\Gamma_y^\nu(q^\nu) \geq 2n\varepsilon$. This motivates the introduction of the list

$$\mathcal{L}_{y,\varepsilon}^\nu = \{q^\nu \in \text{RM}(2, \nu)^\# : \Gamma_y^\nu(q^\nu) \geq 2n\varepsilon\},$$

consisting of every potential prefix of any function $q \in \mathcal{L}_{y,\varepsilon}$. Note also that a function $q^\nu = q^{\nu-1} + x_\nu Q_\nu \in \text{RM}(2, \nu)^\#$ can be in $\mathcal{L}_{y,\varepsilon}^\nu$ only if its prefix $q^{\nu-1}$ is in $\mathcal{L}_{y,\varepsilon}^{\nu-1}$ since, by (7),

$$\begin{aligned} \max_{l \in \text{RM}(1, \nu)} \mathcal{F}(y_s + q^\nu + l) &= \max_{l \in \text{RM}(1, \nu)} (\mathcal{F}(y_{0s} + q^{\nu-1} + l_0) + \mathcal{F}(y_{1s} + q^{\nu-1} + Q_\nu + l_1)) \\ &\leq \max_{l \in \text{RM}(1, \nu-1)} \mathcal{F}(y_{0s} + q^{\nu-1} + l) + \max_{l \in \text{RM}(1, \nu-1)} \mathcal{F}(y_{1s} + q^{\nu-1} + l). \end{aligned}$$

The proposed *sums*-algorithm will recursively determine the intermediate lists $\mathcal{L}_{y,\varepsilon}^v$, for $2 \leq v \leq m$, in the following way: for each $q^{v-1} \in \mathcal{L}_{y,\varepsilon}^{v-1}$, and for each $Q_v \in \text{RM}(1, v-1)^\#$, the corresponding “successor” $q^{v-1} + x_v Q_v$ is tested against the Γ -criterion to decide whether this candidate belongs or not to $\mathcal{L}_{y,\varepsilon}^v$.

To be efficient, the algorithm should generate rather small intermediate lists. In the next section, we give upper bounds for the sizes of the lists $\mathcal{L}_{y,\varepsilon}^v$. In Sect. 4, we give a more precise description of the *sums*-algorithm and give its complexity.

3.1 Size of the intermediate lists

Theorem 3 For $2 \leq v \leq m$, let j such that $1 \leq j \leq \lfloor v/2 \rfloor$. For any vector $y \in \mathbb{F}_2^n$ and any $\varepsilon > 2^{-1-j/2}$, we have

$$|\mathcal{L}_{y,\varepsilon}^v| \leq \frac{\sum_{i=0}^{j-1} A_{2^{v-1}-2^{v-1-i}} (2^{-i} - 2^{-j})}{4\varepsilon^2 - 2^{-j}}$$

Proof Let $L = |\mathcal{L}_{y,\varepsilon}^v|$. For any $q \in \mathcal{L}_{y,\varepsilon}^v$ and for any $s \in \mathbb{F}_2^{m-v}$, let c_s^q be an element of $\text{RM}(1, v)$ such that $\max_{l \in \text{RM}(1, v)} \mathcal{F}(y_s + q + l) = \mathcal{F}(y_s + q + c_s^q)$. Now, let us define the function $h^q : \mathbb{F}_2^v \times \mathbb{F}_2^{m-v} \rightarrow \mathbb{F}_2$ by $h^q(x, s) = q(x) + c_s^q(x)$. Then, by construction, we have

$$2n\varepsilon L \leq \sum_{q \in \mathcal{L}_{y,\varepsilon}^v} \sum_{s \in \mathbb{F}_2^{m-v}} \mathcal{F}(y_s + h_s^q) = \left\langle y_\chi, \sum_{q \in \mathcal{L}_{y,\varepsilon}^v} h_\chi^q \right\rangle.$$

Hence, using the Cauchy–Schwartz inequality, we get that

$$4n^2\varepsilon^2 L^2 \leq \left\langle y_\chi, \sum_{q \in \mathcal{L}_{y,\varepsilon}^v} h_\chi^q \right\rangle^2 \leq \|y_\chi\|^2 \left\| \sum_{q \in \mathcal{L}_{y,\varepsilon}^v} h_\chi^q \right\|^2. \quad (10)$$

Denoting $E_s(q) = \{p + c_s^p + q + c_s^q \in \text{RM}(2, v) : p \in \mathcal{L}_{y,\varepsilon}^v\}$, we get that

$$\mathcal{S}(y) := \left\| \sum_{q \in \mathcal{L}_{y,\varepsilon}^v} h_\chi^q \right\|^2 = \sum_{s \in \mathbb{F}_2^{m-v}} \sum_{q \in \mathcal{L}_{y,\varepsilon}^v} \sum_{f \in E_s(q)} \sum_{x \in \mathbb{F}_2^v} (-1)^{f(x)},$$

with $\sum_{x \in \mathbb{F}_2^v} (-1)^{f(x)} = \mathcal{F}(f) = 2^v - 2\text{wt}(f)$. Then, if we assume that for any $s \in \mathbb{F}_2^{m-v}$ and any $q \in \mathcal{L}_{y,\varepsilon}^v$, the set $E_s(q)$ contains all codewords of weight less than $2^{v-1} - 2^{v-j}$, we know that the remaining codewords in $E_s(q)$ have a weight strictly greater than $2^{v-1} - 2^{v-j}$, that is, at least $2^{v-1} - 2^{v-j-1}$. Hence, using the knowledge of the weight distribution of the $\text{RM}(2, v)$ code, we obtain, for any j such that $1 \leq j \leq \lfloor v/2 \rfloor$, any $q \in \mathcal{L}_{y,\varepsilon}^v$ and any $s \in \mathbb{F}_2^{m-v}$:

$$\sum_{f \in E_s(q)} \mathcal{F}(f) \leq 2^v \sum_{i=0}^{j-1} \frac{A_{2^{v-1}-2^{v-1-i}}}{2^i} + \left(L - \sum_{i=1}^{j-1} A_{2^{v-1}-2^{v-1-i}} \right) 2^{v-j}.$$

Then, by summing over $q \in \mathcal{L}_{y,\varepsilon}^v$ and $s \in \mathbb{F}_2^{m-v}$, we get

$$\mathcal{S}(y) \leq nL \sum_{i=0}^{j-1} \frac{A_{2^{v-1}-2^{v-1-i}}}{2^i} + \left(L^2 - L \sum_{i=0}^{j-1} A_{2^{v-1}-2^{v-1-i}} \right) \frac{n}{2^j}.$$

With the Eq. 10, we get $L \leq \frac{\sum_{i=0}^{j-1} A_{2^{v-1}-2^{v-1-i}}(2^{-i}-2^{-j})}{4\epsilon^2-2^{-j}}$, and the theorem is proved. $\square$

Corollary 2 For any $\epsilon > 0$, let $j \geq 1$ such that $2^{-1-\frac{j-1}{2}} \geq \epsilon > 2^{-1-\frac{j}{2}}$. Then, for any $v \geq 2$ such that $j \leq \lfloor v/2 \rfloor$,

$$|\mathcal{L}_{y,\epsilon}^v| \leq \frac{2^{1+2v(j-1)}}{4\epsilon^2-2^{-j}}.$$

Otherwise, if $j > \lfloor v/2 \rfloor$,

$$|\mathcal{L}_{y,\epsilon}^v| \leq 2^{j(2j-1)}.$$

Proof The first claim is a direct consequence of Corollary 1: for $0 \leq i \leq j-1$, $A_{2^{v-1}-2^{v-1-i}}(2^{-i}-2^{-j}) < A_{2^{v-1}-2^{v-1-i}} \leq 2^{2vi}$, and $\sum_{i=0}^{j-1} 2^{2vi} < 2^{1+2v(j-1)}$. The second inequality comes from the fact that for the first steps v such that $v < 2j$, the list is, in the worst case (i.e. when the list is exhaustive), of size $2^{v(v-1)/2}$. $\square$

Remark 1 Let us denote $\theta = \epsilon - 2^{-1-\frac{j}{2}} > 0$. As $\epsilon \leq 2^{-1-\frac{j-1}{2}}$, we have $\theta \leq 2^{-\frac{j}{2}} \frac{\sqrt{2}-1}{2}$. Then $4\epsilon^2 - 2^{-j} = 4\theta(2^{-j/2} + \theta) = \mathcal{O}(\theta 2^{-j/2})$, and the first inequality of the corollary gives, for fixed θ and j (depending on ϵ), $|\mathcal{L}_{y,\epsilon}^v| = \mathcal{O}(\frac{2^{2v(j-1)+j/2}}{\theta})$. In particular, for $\epsilon > 1/\sqrt{8}$, $|\mathcal{L}_{y,\epsilon}^v| = \mathcal{O}(1/\theta)$ does not depend on v , and $|\mathcal{L}_{y,\epsilon}^v| = \mathcal{O}(2^{2v}/\theta)$ for $\epsilon > 1/4$.

4 Description and complexity of the *sums*-algorithm

In this section, we describe more precisely the *sums*-algorithm, in a recursive manner. In particular, we describe precisely the procedure to follow in order to apply the Γ -criterion, and give the corresponding complexity. As the *sums*-algorithm consists in applying this criterion onto the successors of each element of the intermediate lists, whose sizes were bounded in the previous section, this permits to give the complexity of the algorithm. We also consider the special case of the decoding up to the Johnson bound (Sect. 4.3).

4.1 Recursivity

4.1.1 The intermediate lists

Recall that, at the v -th step, in order to determine $\mathcal{L}_{y,\epsilon}^v$, we have to apply the Γ -criterion onto each element $q^v \in \text{RM}(2, v)^\#$ of the form $q^{v-1} + x_v Q_v$ (for all $Q_v \in \text{RM}(1, v-1)^\#$) whose prefix q^{v-1} is in $\mathcal{L}_{y,\epsilon}^{v-1}$. The problem is that for small values of ϵ , the intermediate list $\mathcal{L}_{y,\epsilon}^v$ may be too big (much bigger than the final list $\mathcal{L}_{y,\epsilon}$) to be stored in memory. That is why we give a recursive description, in the sense that for a given element $q^{v-1} \in \mathcal{L}_{y,\epsilon}^{v-1}$, once all of its valid successors are determined in $\mathcal{L}_{y,\epsilon}^v$, the algorithm chooses one of them, say q^v , and then determines all the valid successors of this q^v before examining another selected candidate in $\mathcal{L}_{y,\epsilon}^{v-1}$. That means that the algorithm will never have to store a complete list $\mathcal{L}_{y,\epsilon}^v$, but only a “partial” list L^v consisting of the linear functions Q_v corresponding to the valid successors of q^{v-1} . Figure 1 represents these lists. As the size $|L^v|$ of L^v is less than (or equal to) 2^{v-1} , and an element Q_v being stored with less than m bits, the total amount of needed memory to manage the lists will be less than $m \sum_{v=2}^m 2^{v-1} < m2^m$ bits. Another non-negligible advantage is that, in this way, we are able to exploit the recursive structure of the $\text{RM}(2, m)$ code to compute the criterion in a faster way.

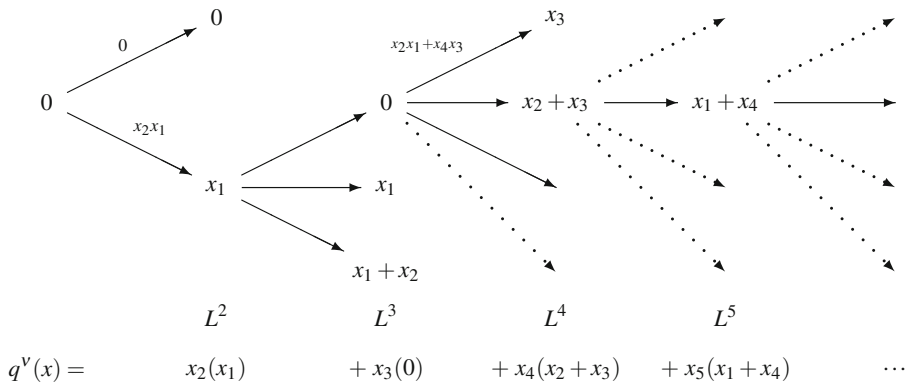


Fig. 1 The sums-algorithm with recursive lists L^v

4.1.2 The Γ -criterion

Let us recall that, in order to apply the Γ -criterion onto each successor $q^v = q^{v-1} + x_v Q_v$ of q^{v-1} , we have to evaluate, for all $s \in \mathbb{F}_2^{m-v}$, the quantity

$$\begin{aligned} \max_{l \in \text{RM}(1, v)} \mathcal{F}(y_s + q^v + l) &= \max_{l \in \text{RM}(1, v)^\#, \delta \in \mathbb{F}_2} \mathcal{F}(y_s + q^v + l + \delta) \\ &= \max_{l \in \text{RM}(1, v)^\#} |\mathcal{F}(y_s + q^v + l)|. \end{aligned}$$

This seems to imply that we need to know the value of $|\mathcal{F}(y_s + q^{v-1} + x_v Q_v + l)|$ for each $Q_v \in \text{RM}(1, v-1)^\#$ and each $l \in \text{RM}(1, v)^\#$. But, in fact, we can do better by separating the character sum according to whether $x_v = 0$ or $x_v = 1$, using (7):

$$\begin{aligned} \max_{l \in \text{RM}(1, v)} \mathcal{F}(y_s + q^v + l) &= \max_{l \in \text{RM}(1, v-1)^\#, l_v \in \mathbb{F}_2} |\mathcal{F}(y_{0s} + q^{v-1} + l) \\ &\quad + \mathcal{F}(y_{1s} + q^{v-1} + Q_v + l + l_v)| \\ &= \max_{l \in \text{RM}(1, v-1)^\#} |\mathcal{F}(y_{0s} + q^{v-1} + l)| \\ &\quad + |\mathcal{F}(y_{1s} + q^{v-1} + Q_v + l)| \end{aligned}$$

The last equality is obtained by fixing l_v so that the two terms have same sign, which obviously gives the greatest value. We now assume that all the values $F_{v-1}[s, u] := \mathcal{F}(y_s + q^{v-1} + u)$, for $s \in \mathbb{F}_2^{m-v+1}$ and $u \in \text{RM}(1, v-1)^\#$ have been stored at the previous step $v-1$ in an array F_{v-1} of size $2^{m-v+1} \times 2^{v-1} = 2^m = n$. Then, denoting $\Gamma^v[Q_v] = \Gamma_y^v(q^{v-1} + x_v Q_v)$ to make it simple, the Γ -criterion can be expressed as:

$$\Gamma^v[Q_v] = \sum_{s \in \mathbb{F}_2^{m-v}} M_s[Q_v] \geq 2n\varepsilon$$

where

$$M_s[Q_v] := \max_{\substack{u, v \in \text{RM}(1, v-1)^\# \\ u+v=Q_v}} |F_{v-1}[0s, u]| + |F_{v-1}[1s, v]|. \quad (11)$$

We see that the complexity to compute $M_s[Q_v]$ for all $Q_v \in \text{RM}(1, v-1)$ is the number of such Q_v 's times the cost of finding a maximum, that is $\mathcal{O}(2^{v-1} \times 2^{v-1}) = \mathcal{O}(2^{2v})$. We deduce

that $\Gamma^v[Q_v]$ can be known for all Q_v with complexity $\mathcal{O}(2^{m-v} \times 2^{2v}) = \mathcal{O}(2^{m+v})$. However, we have introduced the notation M_s because we are able to compute M_s in a much faster way, with a conjectured complexity in $\mathcal{O}(2^v)$. This improvement is discussed in Sect. 4.2. At this point of the recursive algorithm, all the functions Q_v for which $\Gamma^v[Q_v] \geq 2n\varepsilon$ are stored in the list L^v , and the algorithm is run successively at the step $v+1$ for all the corresponding valid candidates $q^{v-1} + x_v Q_v \in \mathcal{L}_{y,\varepsilon}^v$. As we assumed that F_{v-1} was computed at step $v-1$, we need now to compute the array F_v corresponding to the chosen Q_v . The following equalities show that it is done with complexity $\mathcal{O}(n)$. Let $s \in \mathbb{F}_2^{m-v}$ and $u = u^{v-1} + x_v u_v \in \text{RM}(1, v)^\#$. We have:

$$\begin{aligned} F_v[s, u] &= \mathcal{F}(y_s + q^{v-1} + x_v Q_v + u^{v-1} + x_v u_v) \\ &= \mathcal{F}(y_{0s} + q^{v-1} + u^{v-1}) + \mathcal{F}(y_{1s} + q^{v-1} + Q_v + u^{v-1} + u_v) \\ &= F_{v-1}[0s, u^{v-1}] + (-1)^{u_v} F_{v-1}[1s, Q_v + u^{v-1}]. \end{aligned}$$

In the case where $v = m$, then F_m contains the values of $\mathcal{F}(y + q^m + u)$ for all $u \in \text{RM}(1, m)$. If $F_m[u] \geq 2n\varepsilon$ (resp. $\leq -2n\varepsilon$), then $q^m + u$ (resp. $q^m + u + 1$) belongs to $\mathcal{L}_{y,\varepsilon}$.

Note that all the arrays $F_2, \dots, F_{v-1}$ have to be saved simultaneously in memory: once all elements of the current partial list L^v are examined, the next element of the previous list L^{v-1} is examined, using F_{v-2} , and so on. So the memory complexity to manage the recursive arrays F_i is $\mathcal{O}(m^2 2^m)$ bits. Nevertheless, we may prefer an $\mathcal{O}(m 2^m)$ memory complexity (it may be useful for large values of m , e.g. $m \approx 26$) by computing F_v in a non-recursive manner, with a Fast Fourier Transformation (FFT), and with a time complexity in $\mathcal{O}(2^{m-v} \times v 2^v) = \mathcal{O}(v 2^m)$ instead of $\mathcal{O}(2^m)$.

4.1.3 The sums-algorithm

We now resume the recursive version of the *sums*-algorithm. It takes as inputs, at the v -th step, a function $q^{v-1} \in \text{RM}(2, v-1)^\#$, and the corresponding array F_{v-1} (note that the information given by the code-vector y is indirectly contained in F_{v-1}). The output is the set of elements of $\mathcal{L}_{y,\varepsilon}$ admitting q^{v-1} as a prefix.

Algorithm *sums*

input: $\varepsilon, v, m, q^{v-1}, F_{v-1}$

output: valid successors in $\mathcal{L}_{y,\varepsilon}$ of q^{v-1}

- for each $s \in \mathbb{F}_2^{m-v}$, compute M_s from F_{v-1} ,
 - and deduce $\Gamma^v = \sum_s M_s$,
 - and the list $L^v = \{Q_v \in \text{RM}(1, v-1) : \Gamma^v[Q_v] \geq 2n\varepsilon\}$.
 - for each $Q_v \in L^v$ do:
 - compute F_v from F_{v-1} ,
 - if $v < m$ then:
 - call *sums* with input $\varepsilon, v+1, m, q^{v-1} + x_v Q_v, F_v$.
 - if $v = m$ then:
 - for each $u \in \text{RM}(1, m)$,
 - if $|F_m[u]| \geq 2n\varepsilon$, put the corresponding element in $\mathcal{L}_{y,\varepsilon}$.
-

Let us now give the complexity of *sums*. For each $q^v \in \mathcal{L}_{y,\varepsilon}^v$, we need to compute F_v and Γ^{v+1} . The corresponding complexity, in bit operations, is then

$$\underbrace{\mathcal{O}\left(\left|\mathcal{L}_{y,\varepsilon}^v\right|\right)}_{q^v} \times \underbrace{v}_{\text{size in bits}} \times \left(\underbrace{2^m}_{F_v} + \underbrace{2^{m+v}}_{\Gamma^{v+1}}\right).$$

We deduce a total complexity in $\mathcal{O}(\sum_{v=2}^m v \left| \mathcal{L}_{y,\varepsilon}^v \right| 2^m (1 + 2^v))$.

Remark 2 If we compute F_v with a FFT, the term $1 + 2^v$ has to be replaced by $v + 2^v$: the gain does not seem to be really important. But we will see in Sect. 4.2 that in practice, the complexity for the computation of Γ^v is in $\mathcal{O}(2^m)$. Avoiding the FFT, which would be the most costly operation, is then interesting.

Theorem 4 *For any received vector y , the proposed sums-algorithm evaluates the list of all vectors $q \in \text{RM}(2, m)$ such that $d(y, q) \leq n(\frac{1}{2} - \varepsilon)$ with complexity*

$$\mathcal{O}\left(n^2 \log(n) \sum_{v=2}^m |\mathcal{L}_{y,\varepsilon}^v|\right). \quad (12)$$

In particular, denoting $\varepsilon = a + \theta$ with $\theta > 0$, the complexity equals $\mathcal{O}(n^2 \log(n)^2 / \theta)$ for $a = 1/\sqrt{8}$; $\mathcal{O}(n^4 \log(n)^2 / \theta)$ for $a = 1/4$; and $\mathcal{O}(n^{2j+2} 2^{j/2} \log(n)^2 / \theta)$ for $a = 2^{1+j/2}$ and $j \in \{1, \dots, \lfloor \frac{v}{2} \rfloor\}$, fixed correspondingly.

4.2 A sorting technique

The aim of this section is to present an algorithm to compute the quantity $M_s[Q_v]$ (defined in (11)) for all $Q_v \in \text{RM}(1, v-1)^\#$, with a better complexity than $\mathcal{O}(2^{2v})$. Denoting $M = M_s$, $V_0[u] = |F_{v-1}[0s, u]|$ and $V_1[v] = |F_{v-1}[1s, v]|$, the problem is to find

$$M[Q] = \max_{u+v=Q} V_0[u] + V_1[v],$$

for $Q, u, v \in \text{RM}(1, v-1)^\#$. Instead of choosing Q (2^{v-1} possibilities) and computing the corresponding maximum ($\mathcal{O}(2^{v-1})$ complexity), the proposed algorithm runs the greatest values of the set $E = \{V_0[u] + V_1[v] \mid u, v \in \text{RM}(1, v-1)^\#\}$ in the decreasing order until all the $M[Q]$ are known. For such a value $V \in E$, and for each u, v such that $V_0[u] + V_1[v] = V$, if $M[u+v]$ is not already determined, then $M[u+v] = V$. In practice, only $\mathcal{O}(2^v)$ such sums $u+v$ need to be computed. Figure 2 illustrates the values of E for given V_0 and V_1 (corresponding to a random codeword with random noise).

Let us now describe the algorithm. The first observation is that, for $v \geq 3$, all the values $V_0[u]$ (resp. $V_1[v]$) are equal to each other modulo 4. This can be proved by using relation (2) and the fact that the Hamming weight of u is equal to 0 or 2^{v-2} . Thus, for an efficient implementation, it is interesting to divide V_0 and V_1 by 4, without loss of information provided that the sum “rem” of the remainders of this division is stored.

The second observation is that, after this division, the list of values of V_0 (resp. V_1) contains many times almost all the integers between 0 and the maximum $\max_0$ of V_0 (resp. $\max_1$ of V_1). So we can group the functions according to the corresponding value of V_0 (resp. V_1). Let us set

$$P_i[x] = \{u \in \text{RM}(1, v-1)^\# : V_i[u] = x\}, \quad i = 0, 1.$$

Figure 3 illustrates the same array as in Fig. 2 after division by 4 and grouping in P_i .

Now, we obtain the greatest value of E by choosing $u \in P_0[\max_0]$ and $v \in P_1[\max_1]$, and deduce that for those u 's and v 's, $M[u+v] = 4 \times (\max_0 + \max_1) + \text{rem}$. Then, the functions u, v such that $M[u+v] = 4 \times (\max_0 + \max_1 - 1) + \text{rem}$ (the “second” potential value for $M[u+v]$) are obtained by choosing (u, v) either in $P_0[\max_0] \times P_1[\max_1 - 1]$ or in $P_0[\max_0 - 1] \times P_1[\max_1]$. In continuing this process, $M[Q]$ is determined for all Q .

during the decoding of some functions, and divided it by 2^{v-1} . The following array represents the mean of these numbers, for each step v . The Boolean functions f_1 , f_2 , f_3 and f_4 are the received vectors. The decoding radius was chosen equal to the distance of these functions from the $\text{RM}(2, m)$ code.

Step v	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
f_1	1.66	1.750	2.108	2.542	2.361	1.278												
f_2	1.820	1.727	2.029	2.437	3.228	3.917	2.727	1.325										
f_3	1.75	1.92	2.00	1.63	1.45	1.34	1.32	1.40	1.31	1.41	1.43	1.46	1.18	1.45	1.48	1.04		
f_4	1.77	1.94	1.39	1.33	1.30	1.32	1.34	1.34	1.35	1.38	1.38	1.38	1.32	1.37	1.29	1.38	1.002	1.52

The function f_1 (resp. f_2) is the function $\text{trace}(x^{254}) : \mathbb{F}_{2^8} \simeq \mathbb{F}_2^8 \rightarrow \mathbb{F}_2$ in 8 variables (resp. $\text{trace}(x^{1022})$ in 10 variables), it is the trace of the inverse function. It was decoded with $\varepsilon = 0.17969$ (resp. $\varepsilon = 0.11718$). The function f_3 (resp. f_4) is a random quadratic function in 18 (resp. 20) variables, added to an error vector of weight 65,536 (resp. 200,000), corresponding to $\varepsilon = 1/4$ (resp. $\varepsilon = 0.30927$). These experimental results show that in practice, this algorithm appears to be linear in 2^v . However, we could not prove this good complexity.

Conjecture 1 *With this sorting algorithm, the proposed sums-algorithm evaluates, for any received vector y , the list of all vectors $q \in \text{RM}(2, m)$ such that $d(y, q) \leq n(\frac{1}{2} - \varepsilon)$ with complexity $\mathcal{O}(n \log(n) \sum_{v=2}^m |\mathcal{L}_{y, \varepsilon}^v|)$.*

Remark 3 Experimentally, the running time of a decoding has been verified to be proportional to n times the sum of the sizes of the lists.

4.3 Complexity up to the Johnson bound

We give here another method for computing M_s in the case of the decoding up to the Johnson bound, which allows a better complexity than the general case. At the v -th step, we are going to suggest a small list of potential candidates $Q_v \in \text{RM}(1, v-1)^\#$ as continuations. To do this task, we use the classical Plotkin construction: consider the derivative of $f \in \mathcal{B}_m$, defined by $Df(x_1, \dots, x_{v-1}, x_{v+1}, \dots, x_m) = f(x_1, \dots, x_{v-1}, x_v + 1, x_{v+1}, \dots, x_m) + f(x_1, \dots, x_m)$. Note that $\text{wt}(Df) \leq \text{wt}(f)$. Let $q \in \text{RM}(2, m)$ and let $\theta > 0$. If $d(y, q) \leq n(1/2 - (\frac{1}{2\sqrt{2}} + \theta))$, then $d(Dy, Dq) \leq \frac{n}{2}(1 - \frac{1}{\sqrt{2}} - 2\theta) = \frac{n}{2}(\frac{1}{2} - (\frac{1}{\sqrt{2}} - \frac{1}{2} + 2\theta))$. Now we can decode the vector Dy with the list decoding algorithm for the first order RM code of [14], in order to obtain all potential Dq (and hence all potential Q_v). In this case, the complexity is in $\mathcal{O}(n)$ binary operations, and the size of the list is less than or equal to 5, according to the Johnson bound for the $\text{RM}(1, m)$ code. Thus the total cost of these derivative steps is given by $\sum_{v=2}^m \mathcal{O}(n) = \mathcal{O}(n \log_2(n))$. Then, for each $s \in \mathbb{F}_2^{m-v}$, we only need to compute M_s for a small list (of size ≤ 5) of candidates Q_v . This is done in $\mathcal{O}(v2^v 2^{m-v})$ binary operations. We deduce:

Theorem 5 *For any received vector y , the proposed sums-algorithm evaluates the list of all vectors $q \in \text{RM}(2, m)$ such that $d(y, q) \leq n(\frac{1}{2} - \frac{1}{2\sqrt{2}} - \theta)$ with complexity $\mathcal{O}(n \log_2^2(n)/\theta)$.*

We remark that the complexity in [7] was in $\mathcal{O}(n \log_2(n)/\theta^2)$ for the same decoding radius. Hence, for $\theta \leq 1/\log_2(n)$, we improve the previous algorithm.

5 Simulations and implications for the covering radius

We present in this section some experimental results which ensure the efficiency of the *sums*-algorithm.

5.1 The binary symmetric channel (BSC)

We present here the result of the decoding of random quadratic Boolean functions, added to random error vectors. The following figures represent the size of the lists. We took each time the mean over 100 decodings. Figure 4 shows the (logarithm of the) size of each intermediate list, for different values of ε (d is the corresponding decoding radius), generated by the decoding of 9-variable Boolean functions. Figure 5 represents the comparison between the sum of the sizes of the intermediate lists, and the sum of the upper bounds for these sizes given by Corollary 2. Figure 6 represents the quantity $\varepsilon^2 \log_2(\sum_{v=2}^m |\mathcal{L}_{y,\varepsilon}^v|)$, for $m = 8, 9, 10, 12$ and $\varepsilon \in [0.16, \frac{1}{2\sqrt{2}}]$. We see that this quantity is experimentally upper-bounded by 1, that is, $\sum_{v=2}^m |\mathcal{L}_{y,\varepsilon}^v| < 2^{\varepsilon^{-2}}$, and deduce a complexity in $\mathcal{O}(n2^{\varepsilon^{-2}})$ (in bytes operations) for the *sums*-algorithm applied to the BSC.

5.2 Cryptographic functions

We represent here the sizes of the lists at each step $v \geq 5$ of the decoding of some cryptographic functions, which stand for the received vector (the lists for the steps $v = 2, 3, 4$ are exhaustive, of size 2, 8, 64, respectively). These functions are of the form $\text{trace}(x^k) : \mathbb{F}_{2^m} \simeq \mathbb{F}_2^m \rightarrow \mathbb{F}_2$, for some integer k . The decoding radius d (associated with ε) was chosen as small as possible so that solutions can be found (except for the Welch function in 13 variables). The number t of needed hours for the computation (with a PC at 3 GHz) is also displayed.

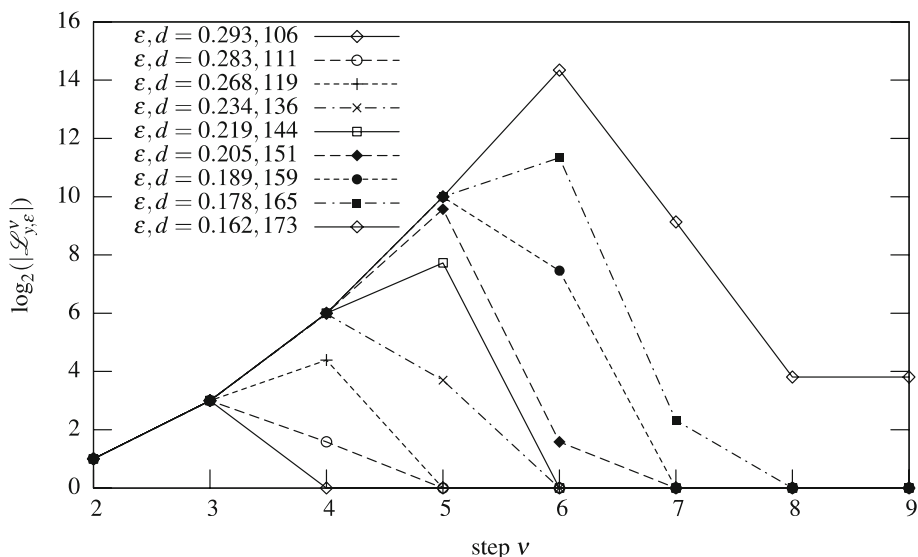


Fig. 4 Size of the intermediate lists ($m = 9$), at each step, for different values of ε , in the BSC

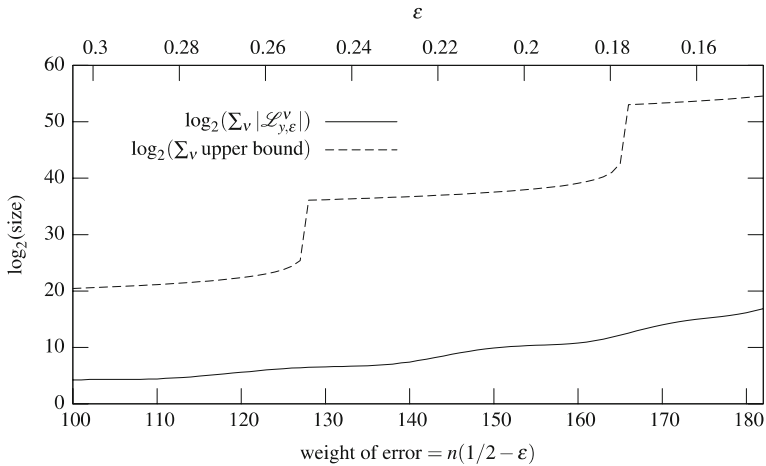


Fig. 5 Sum of the sizes of the intermediate lists ($m = 9$, in the BSC), for different values of ε , compared with theoretical bounds of Corollary 2

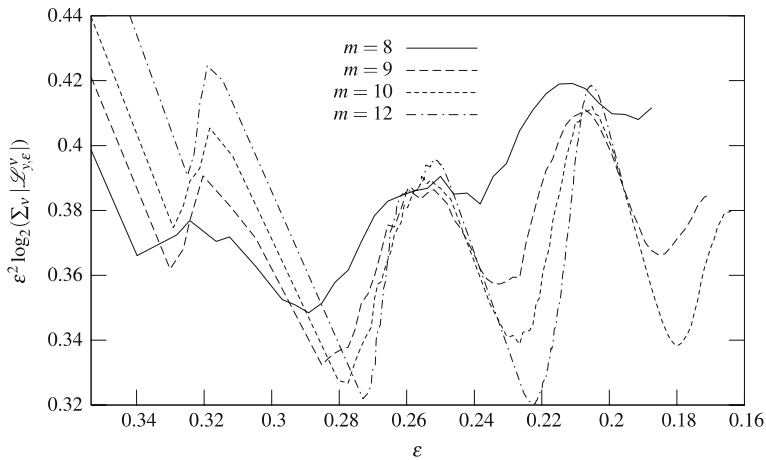


Fig. 6 The quantity $\varepsilon^2 \log_2(\sum_{v=2}^m |\mathcal{L}_{y,\varepsilon}^v|)$ for different m and ε : experimentally upper-bounded by 1 (in the BSC)

5.2.1 The Welch function ($\text{trace}(x^{2^{(m-1)/2}+3})$)

$m \setminus \text{Step } v$	5	6	7	8	9	10	11	12	13	d	ε	t
7	256	512	1,027							36	0.219	
9	1,024	31,232	209,408	22,016	23,552					184	0.141	
11	1,024	32,768	2,097,152	254,615,552	119,707,648	1,603,584	1,628,160			848	0.086	7
13	1,024	32,768	2,097,152	268,435,456	1,592,680,448	0	0	0	0	3,487	0.0743	149

We see that the decoding of this function in $\text{RM}(2, 13)$ with $d = 3,487$ does not give any solution. However a solution was found at distance 3,632. We deduce that $3,487 < d(x^{67}, \text{RM}(2, 13)) \leq 3,632$. We also obtained that $d(x^{131}, \text{RM}(2, 15)) \leq 15,488$ and $d(x^{259}, \text{RM}(2, 17)) \leq 63,680$.

5.2.2 The inverse function ($\text{trace}(x^{2^m-2})$)

$m \setminus v$	5	6	7	8	9	10	11	12	d	ε	t
7	448	76	78						36	0.219	
8	1,023	4,495	16	16					82	0.180	
9	1,024	32,666	38,285	9	9				182	0.145	
10	1,024	32,768	2,016,990	68,303	2	2			392	0.117	0.03
11	1,024	32,768	2,097,152	263,217,917	4,492,858	66	66		842	0.089	6
12	1,024	32,768	2,097,152	268,435,456	52,637,907,827	7,139,776	6	6	1,760	0.0703	≈ 90 days

For $m = 13, 14$ and 15 , we have $d(\text{trace}(x^{8,190}), \text{RM}(2, 13)) \leq 3,696$,
 $d(\text{trace}(x^{16,382}), \text{RM}(2, 14)) \leq 7,580$ and $d(\text{trace}(x^{32,766}), \text{RM}(2, 15)) \leq 15,506$.

5.2.3 The Kasami functions ($\text{trace}(x^{2^{2k}-2^k+1})$, with $\gcd(m, k) = 1, k \leq m/2$)

$m; k \setminus \text{Step } v$	2	3	4	5	6	7	8	9	10	11	d	ϵ
7;3	2	8	64	39	7	7					32	0.25
8;3	2	8	64	198	112	56	68				72	0.219
9;4	2	8	64	1,024	28,393	4,622	137	141			176	0.156
10;3	2	8	64	1,024	32,768	1,201,601	7,144	165	165		384	0.125
11;4	2	8	64	1,024	32,768	2,097,152	81,407,968	47,918	99	99	824	0.098

5.3 The covering radius of the second order Reed–Muller codes

5.3.1 Known results

Despite that the Reed–Muller codes have been studied for many years, their exact covering radius are yet unknown. First, let us recall that the covering radius $\rho(2, m)$ of the $\text{RM}(2, m)$ code is defined by $\rho(2, m) = \max_{y \in \mathcal{B}_m} d(y, \text{RM}(2, m))$, where $\mathcal{B}_m$ is the set of all Boolean functions. Concerning the upper bound for $m \geq 10$, the best known result comes from [8] and states:

Theorem 6 ([8]) *For every positive integer $m \geq 10$, one has*

$$\rho(2, m) \leq \left\lfloor 2^{m-1} - \frac{\sqrt{15}}{2} \cdot 2^{\frac{m}{2}} \cdot \left(1 - \frac{122,929}{21 \cdot 2^m} - \frac{155,582,504,573}{4,410 \cdot 2^{2m}}\right) \right\rfloor$$

The following result, from [10] (an exact version of the asymptotic lower bound given in [9]), gives a lower bound for the covering radius:

Theorem 7 ([10]) *Let c be a positive real number such that $c > \sqrt{\log(2)}$. Then*

$$\rho(2, m) \geq 2^{m-1} - c \sqrt{1 + m + \frac{m(m-1)}{2}} 2^{\frac{m-1}{2}}.$$

We summarize in the following table the best known bounds for small values of m (see [8–10, 15, 16]):

m	2	3	4	5	6	7	8	9	10	11	12
Lower bounds of $\rho(2, m)$	0	1	2	6	18	40	84	171	372	806	1,714
Upper bounds of $\rho(2, m)$	0	1	2	6	18	44	100	220	464	956	1,946

We have proposed in this article a strong list decoding algorithm, which permits to improve the lower bounds on the covering radius of the second order Reed–Muller code for $9 \leq m \leq 12$.

5.3.2 Lower bound on the covering radius

In order to try to lower bound the covering radius for a small number of variables, we tested, for $m \leq 10$, all power functions of the form $\text{trace}(x^k)$ and all sums of two power functions, that is, functions of the form $\text{trace}(x^k + x^h)$ defined over $\mathbb{F}_{2^m}$ with $k, h < 2^m$. We have displayed in the following table the Hamming distance of the furthest power functions we found from the $\text{RM}(2, m)$ codes. We compare the values obtained with the lower bounds given in [9] and [10].

m	f	$d(f, \text{RM}(2, m))$	Bounds of [9]
6	$x^{21}; x^{11} + x^{21}; x^{13} + x^{21}; x^{21} + x^{27}$	18	18 (exact)
7	$x^7 + x^{15}$	38	40
8	$x^7; x^{73}$	84	84
9	x^{73}	196	171
10	$x^7; x^{35}; x^{37}; x^{41}; x^{49}; x^{73}; x^{85}$	400	372
11	$x^{67}; x^{73}$	848	806
12	$x^{4,094}$	1,760	1,714

For 7 variables, we did not succeed to find such far Boolean function ($d = 40$).

In the following table, we represent the values of $\frac{2^{m-1}-d}{2^{\frac{m-1}{2}} \times \sqrt{1+m+\frac{m(m-1)}{2}}}$, related to Theorem 7, which have to be compared to $\sqrt{\log(2)} \approx 0.833$.

m	4	5	6	7	8	9	10	11	12
d	2	6	18	40	84	196	400	848	1,760
$\frac{2^{m-1}-d}{2^{\frac{m-1}{2}} \times \sqrt{1+m+\frac{m(m-1)}{2}}}$	0.64	0.63	0.53	0.56	0.64	0.55	0.66	0.67	0.72

6 Conclusion

We have proposed a list decoding algorithm for the second order Reed–Muller code and studied its complexity. According to the results of our simulations, we see that the complexity that we have conjectured makes sense. This low complexity allowed to consider the computation of some lower bounds for the covering radius. We do not know yet if the bounds concerning the size of the list beyond the Johnson radius that were given in [6] are tight or not. It would be very interesting to construct some Boolean functions which are optimally far from the second order Reed–Muller codes of higher orders; this is an open problem.

An interesting question concerns the extension of this algorithm to Reed–Muller codes of order three. The main obstacle comes from the fact that running over all elements of the second order Reed–Muller code is expensive. The reason is that the dimension of this code is huge. We do not have yet any idea of how to proceed.

We have not considered the case of the generalized Hamming distance, but this problem should not be so difficult; nevertheless the size of the intermediate lists should be recalculated, so this problem is worth studying.

The case of the binary symmetric channel is also very interesting according to the simulation results, and it merits a strong theoretical study.

Acknowledgements We gratefully thank Professor Grigory Kabatiansky and Professor Claude Carlet for very helpful and inspiring discussions. The work of C. Tavernier was supported by DISCREET, IST project no. 027679, funded in part by the European Commission's Information Society Technology 6th Framework Program.

References

1. Elias P.: List decoding for noisy channels. 1957-IRE WESCON Conv. Record **2**, 94–104 (1957).
2. Guruswami V., Sudan M.: Improved decoding of Reed-Solomon and algebraic-geometry codes. *IEEE Trans. Inform. Theory* **45**, 1757–1767 (1999).
3. Goldreich O., Levin L.A.: A hard-core predicate for all one-way functions. In: *Proceedings of 21st ACM Symp. on Theory of Computing*, pp. 25–32 (1989).
4. Goldreich O., Rubinfeld R., Sudan M.: Learning polynomials with queries: the highly noisy case. In: *Proceedings of 36th Symp. on Foundations of Computer Science*, pp. 294–303 (1995).
5. Pellikaan R., Wu X.-W.: List decoding of q -ary Reed–Muller Codes. *IEEE Trans. Inform. Theory* **50**, 679–682 (2004).
6. Kabatiansky G., Tavernier C.: List decoding of second order Reed–Muller Codes. In: *Proceedings of the 8th Intern. Simp. Comm. Theory and Applications*, Ambleside, UK, July 2005.
7. Dumer I., Kabatiansky G., Tavernier C.: List decoding of Reed–Muller codes up to the Johnson bound with almost linear complexity. In: *Proceedings of the ISIT 2006*. Seattle, USA (2006).
8. Carlet C., Mesnager S.: Improving the upper bounds on the covering radii of binary Reed–Muller codes. *IEEE Trans. Inform. Theory* **53**, 162–173 (2007).
9. Cohen G., Honkala I., Litsyn S., Lobstein A.: *Covering Codes*. North-Holland (1997).
10. Carlet C.: The complexity of Boolean functions from cryptographic viewpoint. In: Krause M., Pudlak P., Reischuk R., van Melkebeek D. (eds.) *Dagstuhl Seminar “Complexity of Boolean Functions”*, 15 pp. (2006).
11. Dumer I.: Recursive decoding and its performance for low-rate Reed–Muller codes. *IEEE Trans. Inform. Theory* **50**, 811–823 (2004).
12. Sloane N., Berlekamp E.: Weight enumerator for second-order Reed–Muller codes. *IEEE Trans. Inform. Theory* **16**(6), 745–751 (1970).
13. MacWilliams F.J., Sloane N.J.A.: *The Theory of Error-Correcting Codes*. North-Holland Publishing Company (1977).
14. Kabatiansky G., Tavernier C.: List decoding of first order Reed–Muller codes II. In: *Proceedings of the ACCT-10*. Zvenigorod, Russia (2006).
15. Hou X.D.: Some results on the covering radii of Reed–Muller codes. *IEEE Trans. Inform. Theory* **39**(2), 366–378 (1993).
16. Schatz J.: The second-order Reed–Muller code of length 64 has covering radius 18. *IEEE Trans. Inform. Theory* **IT-27**(4), 529530 (1981).